Лабораторна робота №7

Мета роботи: Сформувати практичні навички роботи з відеопотоком у ROS2, навчитися отримувати зображення з камери, виконувати базову обробку за допомогою OpenCV та інтегрувати комп'ютерний зір у робототехнічну систему.

Посилання на репозиторій: [Anastasia11070830/ROS2]

Хід роботи

1. Працюємо в тому самому врітуальному середовищі.
2. **Підготовка середовища**

Робимо налаштування терміналу для роботи з ROS 2 та перевірка базового стану системи.

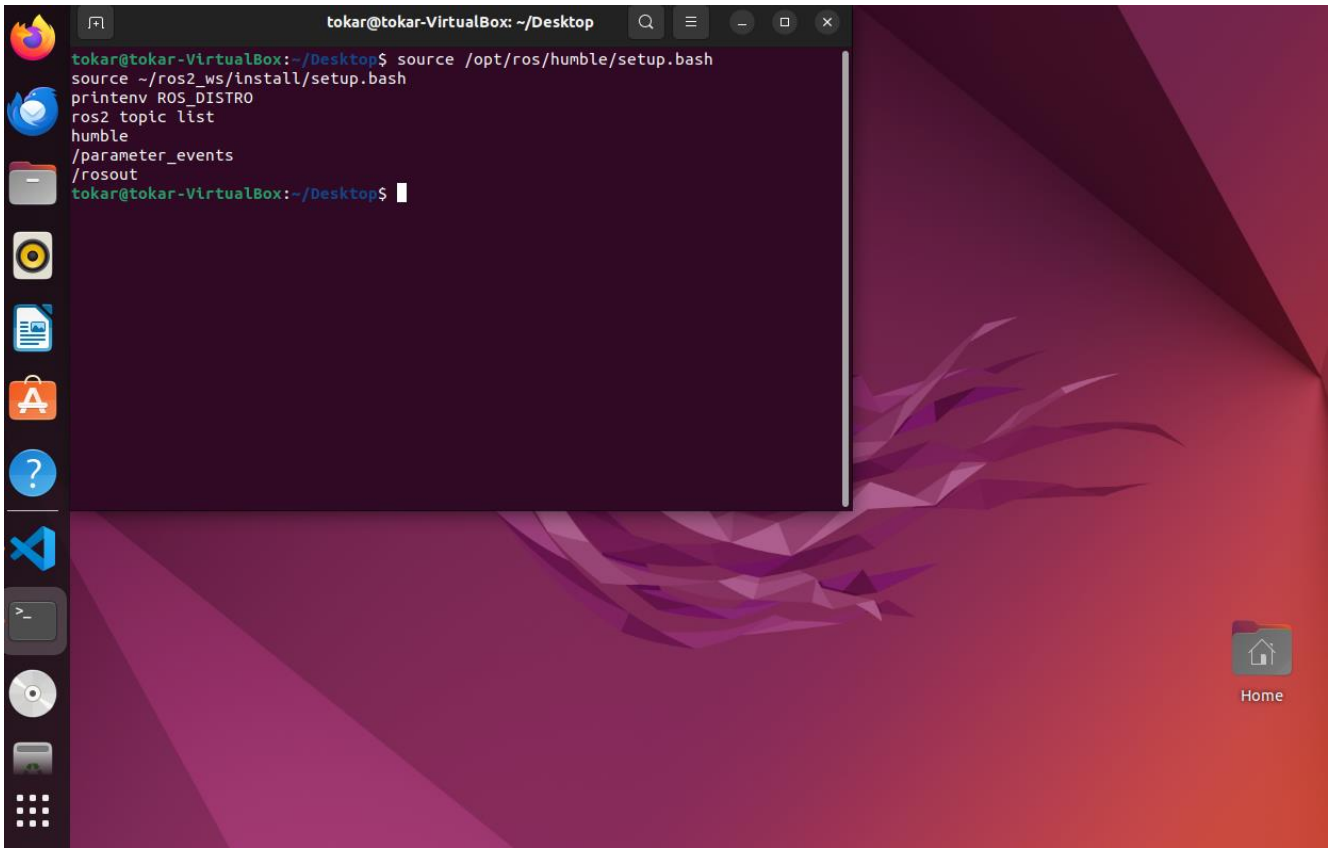


Рисунок 1 – Підготовка середовища

3. **Створення пакету із залежностями.**
   У результаті виконання команд було успішно згенеровано пакет vision_pkg зі стандартною структурою. Отриманий вивід команди tree підтверджує наявність усіх необхідних конфігураційних файлів (package.xml, setup.py, setup.cfg), директорій для тестів та ресурсів, а також безпосередньо вихідного файлу vision_node.py. Робоче середовище готове до написання прикладного коду.

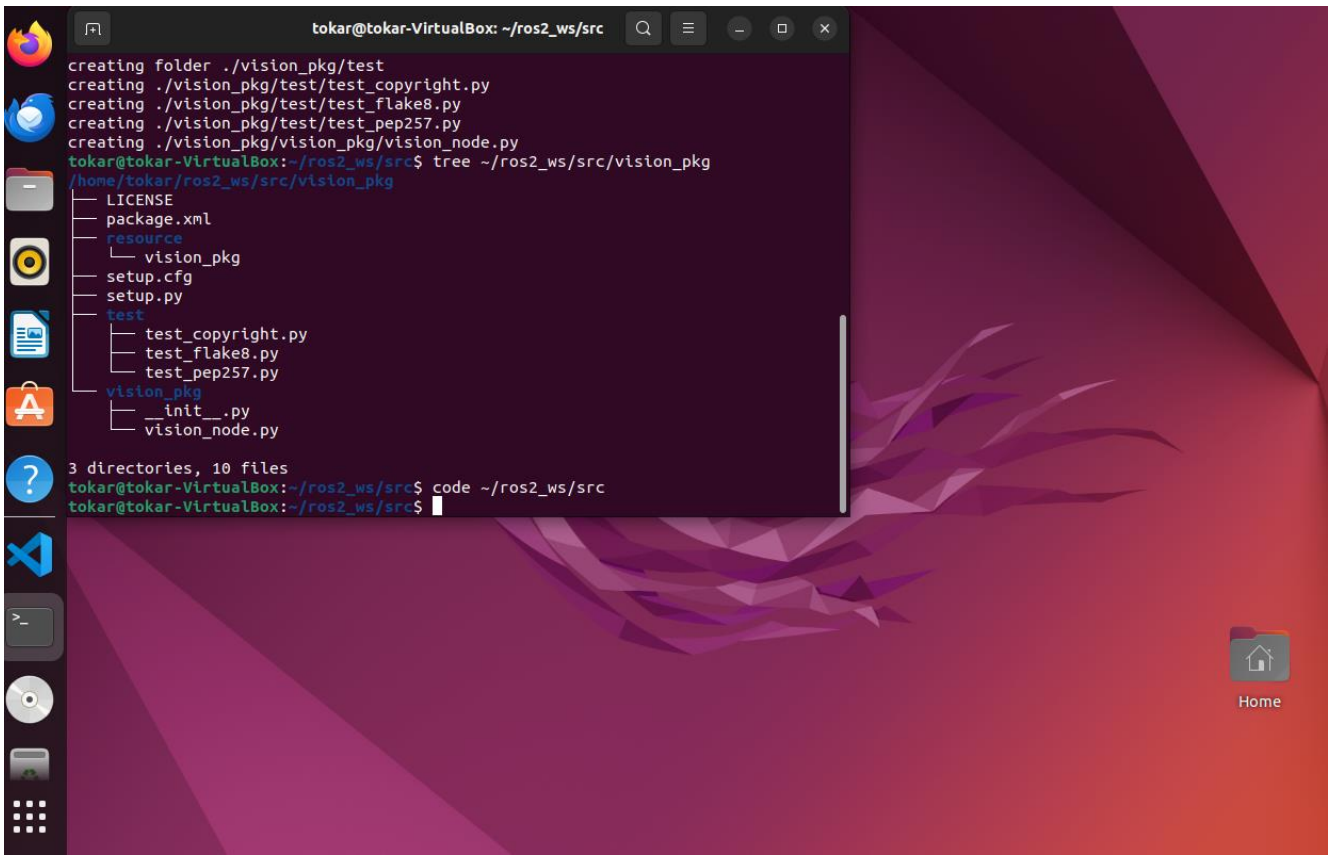


Рисунок 2 -створення пакету

4. Залежність від OpenCV у package.xml

Налаштування маніфесту пакету для використання бібліотеки комп'ютерного зору OpenCV. У файл package.xml успішно внесено системну залежність python3-opencv. Це гарантує, що середовище ROS 2 коректно ідентифікує необхідність наявності бібліотеки OpenCV для подальшого написання та запуску коду обробки відеопотоку.

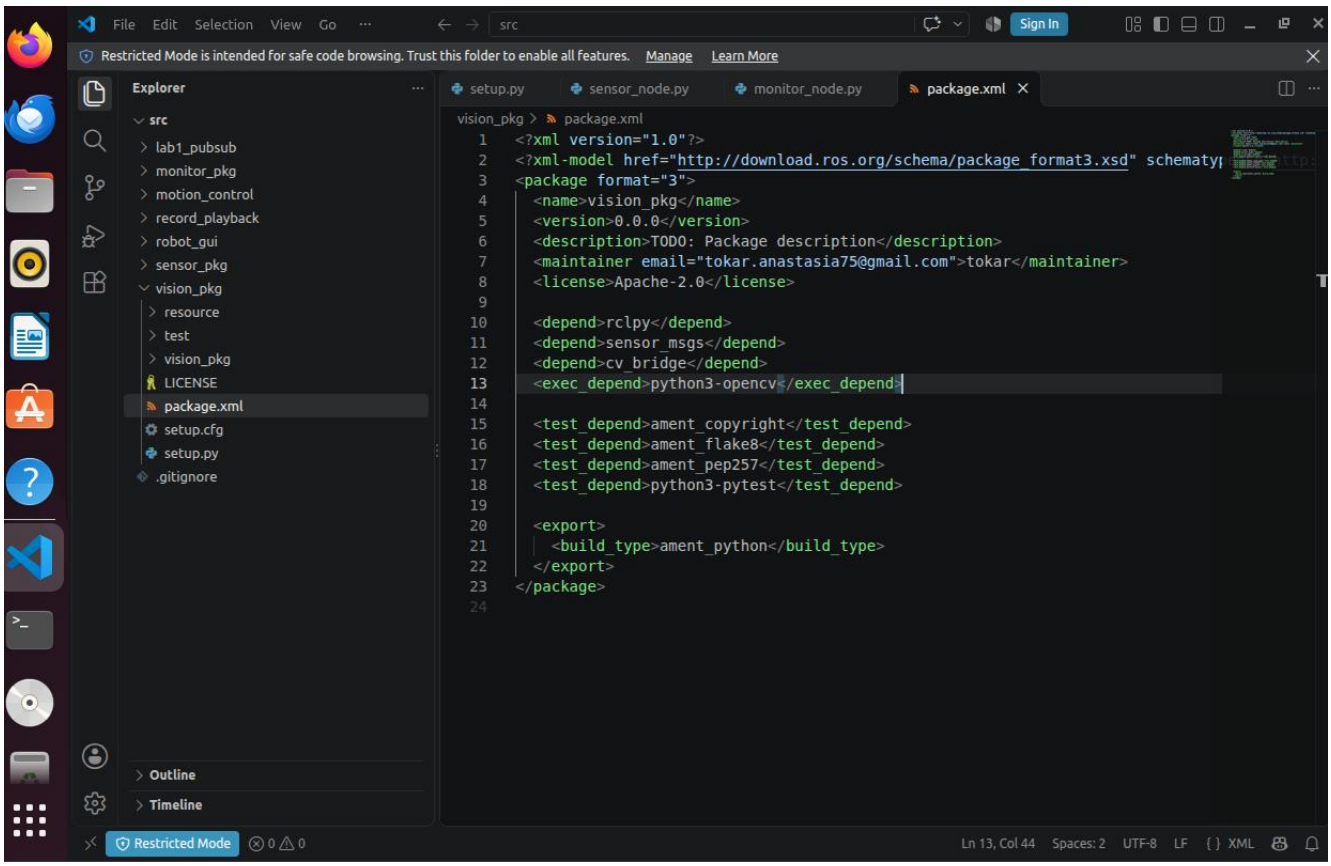


Рисунок 3 – демонстрація коду у VS code

5. Симуляція камери і пошук топіка камери

На даному етапі ми створюємо програму (вузол ROS 2), яка вигадує (генерує) картинки за допомогою Python та OpenCV і транслює їх у мережу ROS так, ніби це справжнє відео з камери.

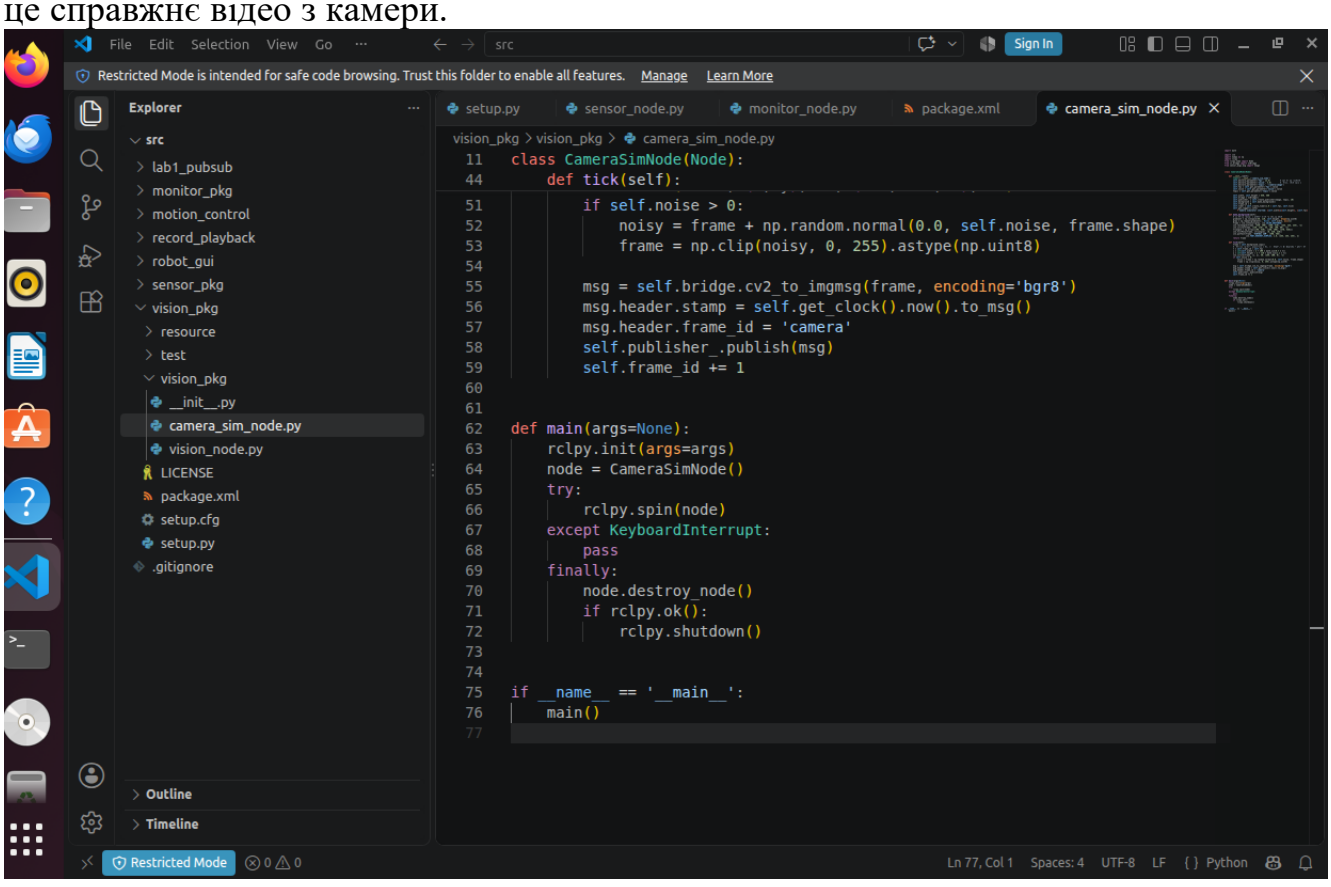


Рисунок 4 – створення «камери»

6. Реєстрація в setup.py та Збірка

На цьому етапі необхідно зробити дві важливі речі: зареєструвати нову програму (симулятор камери) в системі, щоб ROS 2 знав, як її запускати, і збираєте (компілюєте) весь пакет, щоб перевірити, що все працює.

Перевірка показала, що система коректно розпізнає обидва вузли: vision_pkg camera_sim_node та vision_pkg vision_node. Середовище повністю готове до запуску симулятора камери.

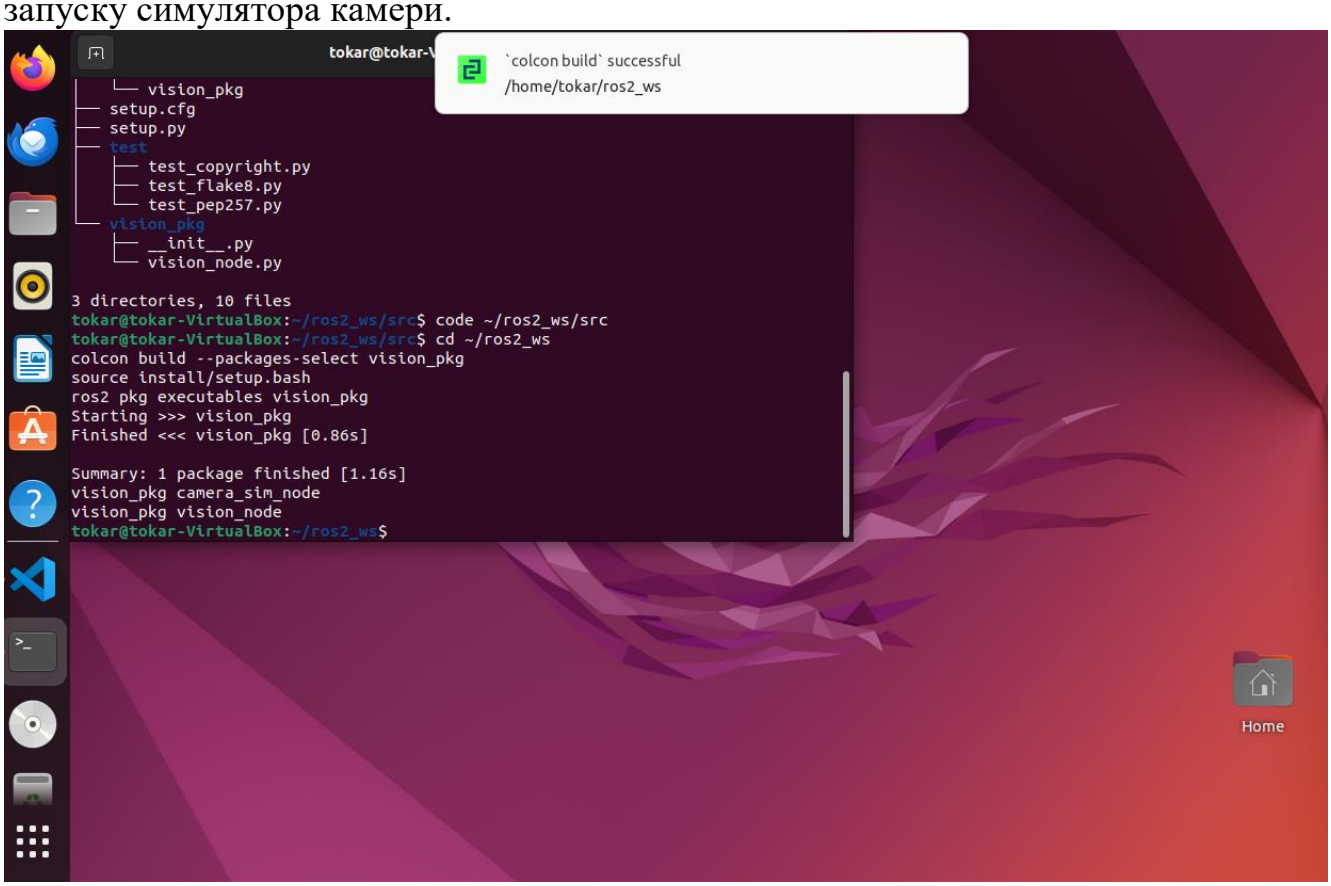


Рисунок 5 – підготовка середовища то запуску симулятора

7. Запуск камери і пошук топіка

На цьому етапі запускаємо розроблений вузол симулятора камери та за допомогою консольних утиліт ROS 2 перевірити наявність топіка, частоту публікації та структуру повідомлень із зображенням. Перевірка підтвердила, що він коректно публікує зображення у топік /camera/image. Аналіз отриманого повідомлення показав, що зображення має роздільну здатність 640x480, кодування кольору bgr8 та коректно заповнений заголовок (header) із міткою часу. Система готова до розробки вузла-споживача (обробника зображень).

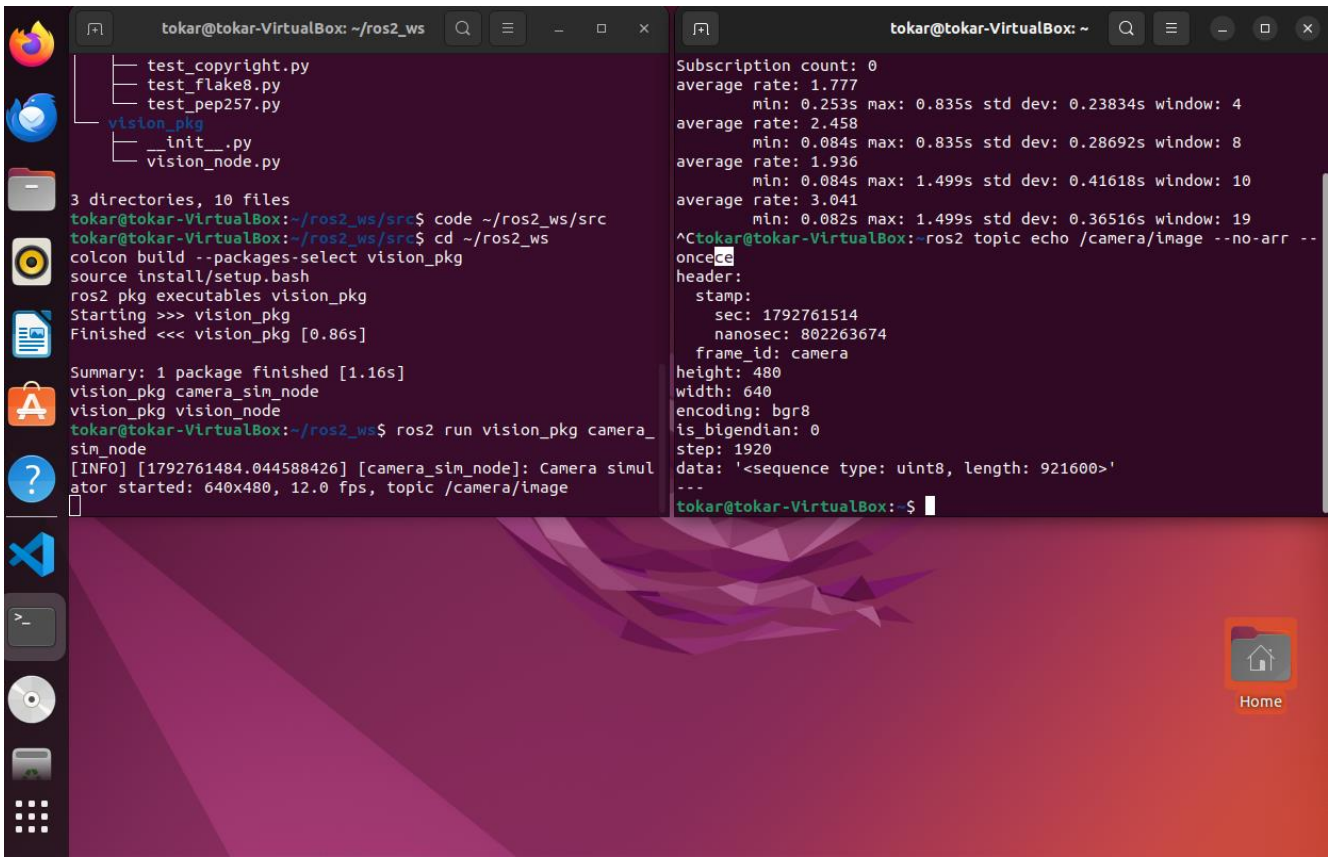


Рисунок 6 – демонстрація що система готова до розробки

8. Перегляд відео через rqt_image_view

Необхідно переконатися в тому, що вузол-симулятор коректно генерує та публікує зображення, шляхом візуалізації даних із топіка за допомогою стандартних інструментів ROS 2. Зображення з топіка /camera/image коректно відображається стороннім інструментом, що свідчить про правильність формування повідомлень типу sensor_msgs/msg/Image та готовність системи до розробки вузла комп'ютерного зору (vision_node).

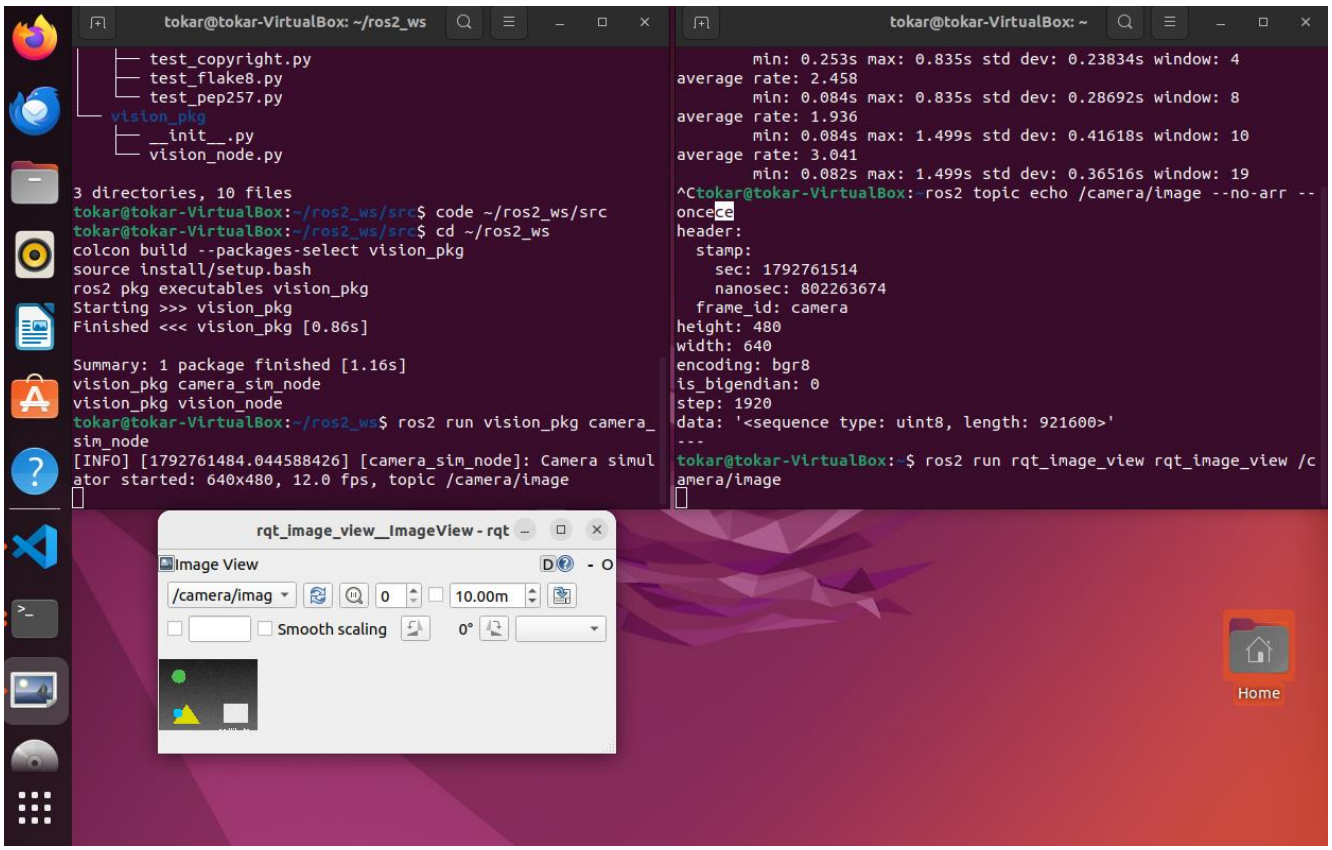


Рисунок 7 – Перегляд відео

9. Код Vision Node

Створено повноцінний вузол vision_node, який готовий приймати відеопотік, аналізувати його за заданими параметрами, зберігати результати проміжної обробки на диск та візуалізувати роботу алгоритмів у реальному часі.

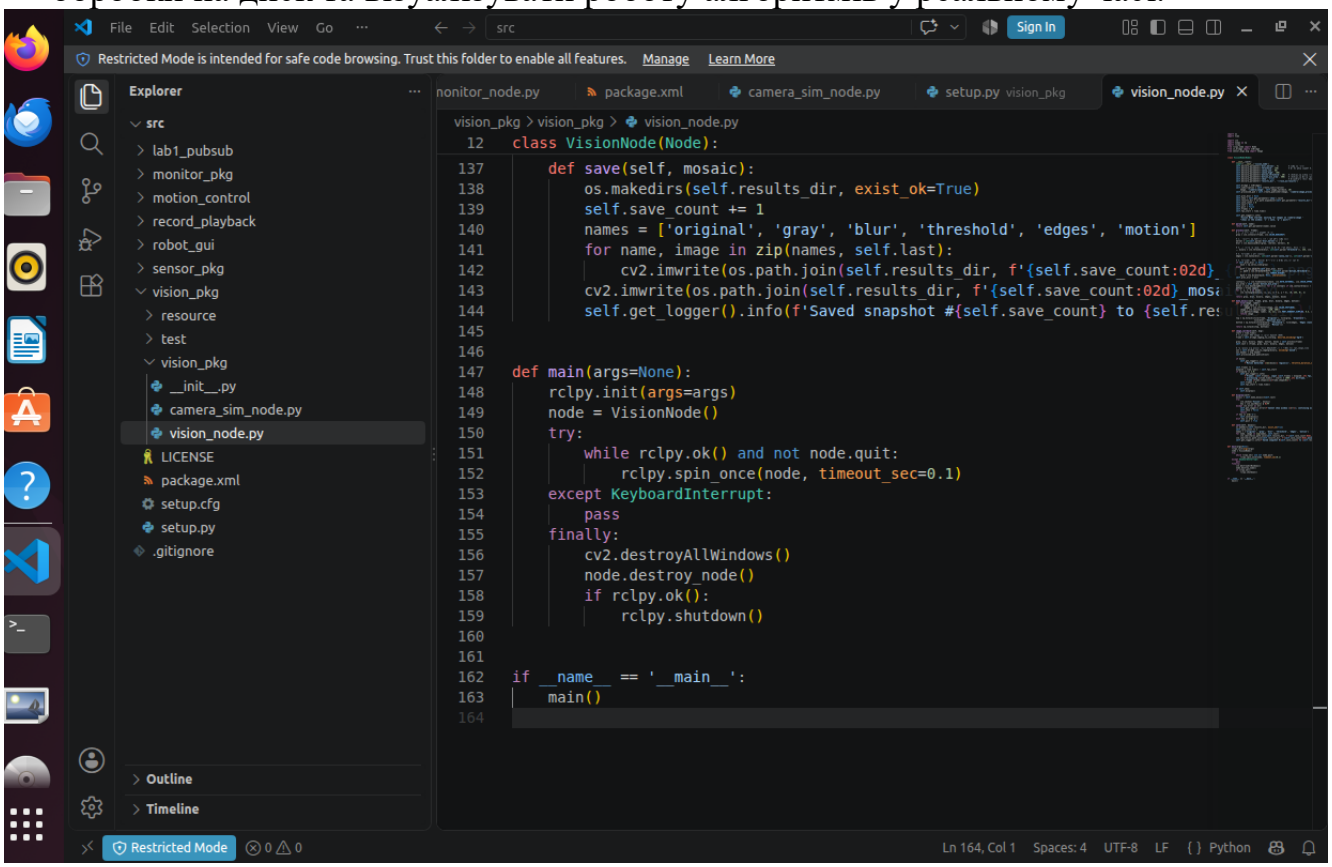


Рисунок 8– Демонстрація VS code

10.Запуск камери і Vision Node

Перевіряємо спільну роботу вузла-симулятора камери та вузла комп'ютерного зору, а також оцінити якість та швидкість роботи реалізованих алгоритмів обробки в реальному часі. Вузол комп'ютерного зору успішно приймає дані від симулятора, виконує повний цикл обробки зображень (від фільтрації до детекції руху) та візуалізує результати в реальному часі. Швидкодія алгоритмів є достатньою для обробки відеопотоку без затримок.

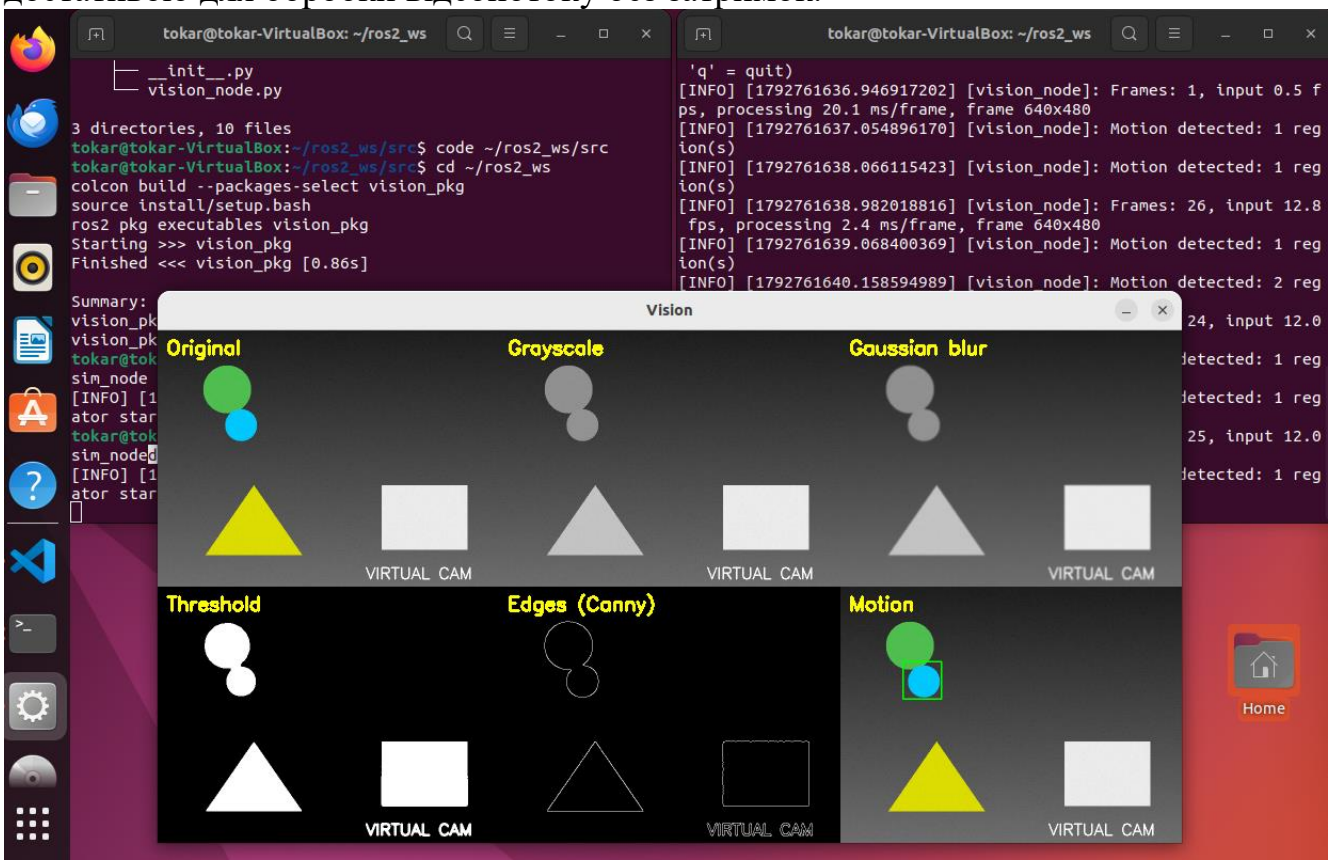


Рисунок 9 – Запуск камери

11. **Динамічна зміна параметрів обробки зображень**

Дослідити можливість зміни параметрів роботи вузла в реальному часі без необхідності його перезапуску або повторної збірки, та оцінити вплив цих параметрів на результат комп'ютерного зору. Підтверджено ефективність архітектури ROS 2 щодо динамічного конфігурування вузлів. Зміна параметрів threshold та blur_kernel через консоль миттєво впливає на алгоритми обробки зображень, що дозволяє гнучко налаштовувати систему під різні умови освітлення та типи об'єктів без втручання в програмний код. Згенеровані зображення успішно збережено на диск.

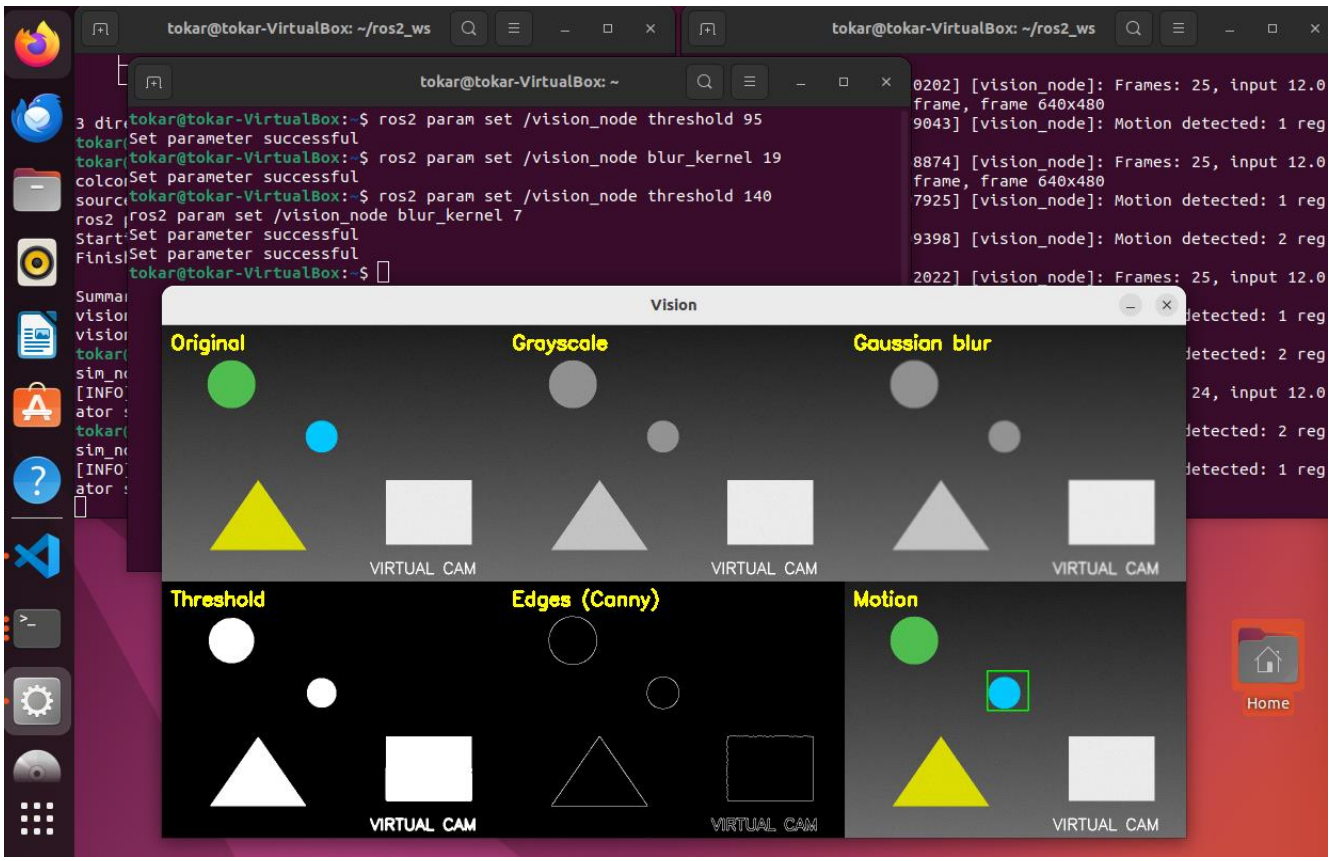


Рисунок 10 – Зміна параметрів

12.Оброблене відео як топік

Утиліта rqt_image_view успішно відобразила чорно-біле (бінаризоване) зображення з топіка /camera/image_processed. Це підтверджує, що розроблений вузол vision_node коректно виконує роль видавця (publisher) для оброблених даних. Такий підхід дозволяє інтегрувати створений модуль обробки зображень у складніші робототехнічні системи, де інші вузли (наприклад, алгоритми навігації або розпізнавання об'єктів) можуть незалежно підписуватися на цей топік.

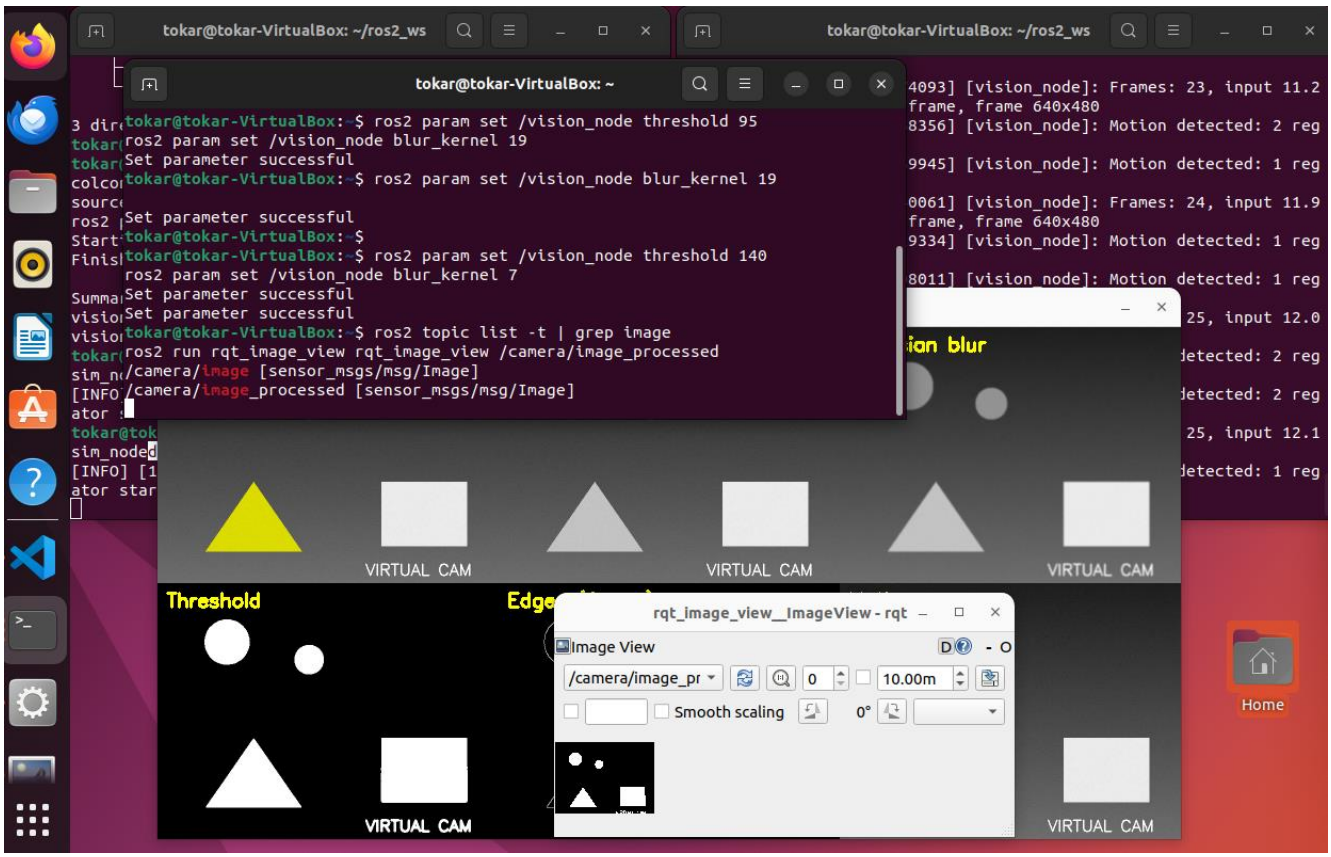


Рисунок 11 – Оброблене

Висновок: У ході лабораторної роботи було успішно розроблено та протестовано систему комп'ютерного зору на базі ROS 2 та бібліотеки OpenCV. Було створено власний пакет vision_pkg, реалізовано вузол-симулятор камери для генерації відеопотоку та основний вузол обробки зображень із використанням інструменту cv_bridge. Реалізовано повний конвеєр обробки кадрів, що включає переведення у відтінки сірого, гаусове розмиття, бінаризацію, виділення контурів (Canny) та детекцію руху. Експериментально доведено можливість динамічної зміни параметрів вузла «на льоту» без перезапуску програми, а також налаштовано публікацію оброблених зображень в окремий топік для подальшого використання іншими компонентами. Мета роботи досягнута повністю: система працює в реальному часі, а отримані результати підтверджують ефективність архітектури ROS 2 для побудови гнучких та модульних систем технічного зору.